
TECHNICAL DOCUMENTATION

GridSec Sim

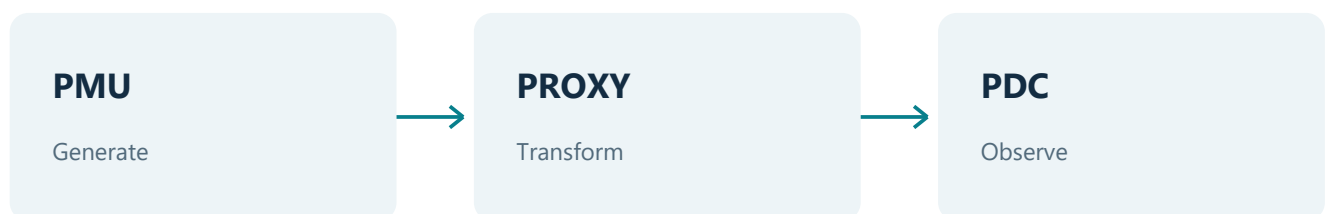
Software documentation

A complete guide to installation, experiments, architecture, protocols, integrations, and project maintenance.

APPLICATION 1.0.0

Project GitHub

<https://github.com/neerajjayesh/GridSec-project>



Research and laboratory use. This manual describes implemented behavior and confirmed limitations; it does not claim production readiness or protocol certification.

Contents

This manual consolidates the current project documentation. Click a chapter below or use the PDF bookmarks. Companion API/schema/example files are embedded as PDF attachments.

01 Product overview	3
02 Getting started	5
03 User guide	7
04 Architecture	10
05 Configuration reference	14
06 Attack engine reference	17
07 Protocol reference	20
08 REST API reference	23
09 Integrations and exports	27
10 Data formats	30
11 Operations and troubleshooting	33
12 Security model and reporting	37
13 Developer guide	38
14 Contributing	41
15 Testing and verification	42
16 Known limitations	46
17 Glossary	48
18 Changelog	50
19 Reference files	51
20 Practical examples	52

Commands run from the repository root unless stated otherwise. Example measurements are illustrative. Source references name repository files; the PDF does not embed application source. Earlier scenario notes remain historical and are superseded by the current guides.

CHAPTER 01

01 Product overview

Project GitHub: [neerajjayesh/GridSec-project](https://github.com/neerajjayesh/GridSec-project)

Purpose and users

GridSec Sim is a visual laboratory for studying how changes to measurement traffic affect data visible to a downstream receiver. Operators create a substation diagram, start a synthetic PMU, select a stream modification, and inspect resulting measurements and packet classifications.

User	Typical work
Instructor or student	Demonstrate clean traffic, false measurements, stale data, delay, and loss
Security researcher	Inspect controlled protocol changes and collect synthetic experiment records
Integration developer	Exercise a laboratory receiver or prototype an export consumer
Contributor	Extend codecs, improve runtime wiring, and maintain the desktop application

The project focuses on measurement communication. It does not solve electrical power flow, model a physical grid, or implement a complete substation control system. A frequency override changes the reported field; it does not change a physical generator model.

Functional scope

Area	Current implementation
Desktop	PyQt6 canvas, property editor, attack panel, plots, packet log, and integration tabs
Topologies	Thirteen node types, four levels, protocol links, Threat Agent attachments, JSON persistence
Validation	Missing PMU/Local PDC errors; warnings for missing C37.118 links, unattached agents, and duplicate PMU ports
Main runtime	One UDP PMU and proxy, using the first PMU and Local PDC
Attacks	Ten selectable modules; one active module at a time; optional frame window
Visualization	Original/modified phase-A magnitude, frequency, and phase-A angle
Auxiliary traffic	DNP3/UDP, Modbus/TCP, and a GOOSE raw Ethernet publisher attempt
IEC104	Frame helpers and diagram labels; no desktop live simulator
Records	In-memory incidents, recent packet metadata, JSON/CSV, experimental STIX
External tooling	Core PCAP, Syslog/CEF, and REST components with desktop wiring gaps

Experiment outcomes

A false-data experiment establishes a clean baseline, selects a known transformation, observes the modified proxy record, and verifies receipt independently when downstream delivery matters. For example, SCALE with `scale_factor=2.0` changes a reported 120 V magnitude to 240 V.

The plot observes proxy processing. It does not prove that a remote PDC received, accepted, or acted on the outgoing frame. Loss and latency need receiver-side or network evidence in addition to plots.

Deployment and quality characteristics

Characteristic	Current position
Execution	One local Python process, worker threads, Qt event loop
Storage	Topology/export files and bounded memory; no database
Distribution	Source checkout and Python requirements; no packaged executable
Reproducibility	Minimum dependency versions; no lockfile or GUI seed setting
Capacity	One desktop C37.118 stream; no measured production throughput envelope
Availability	No supervisor, cluster, failover, or durable event queue
Identity	Optional bearer API token; no accounts or roles

Development direction

Identified needs include integration lifecycle wiring, packet-byte capture, codec ownership, and explicit UI-to-runtime configuration mapping. Per-link execution requires separate stream contexts, codec and attack state isolation, and concurrency-safe integration handling. These are engineering needs, not a committed roadmap.

Source: main window (gui/main_window.py), attack engine (core/attack_engine.py), integration manager (core/integration_manager.py).

CHAPTER 02

02 Getting started

Obtain the source

The project repository is [neerajjayesh/GridSec-project](https://github.com/neerajjayesh/GridSec-project).

TEXT

```
git clone https://github.com/neerajjayesh/GridSec-project.git
cd GridSec-project
```

If you already have the source, open the directory containing `main.py` and continue with the installation steps below.

Requirements

Requirement	Details
Python	Project baseline: 3.10+. Review environment: Windows with Python 3.14.6.
Display	Qt-compatible desktop; minimum window 1200 × 700, initial size 1600 × 950
Packages	Install all entries in <code>requirements.txt</code>
Source	Keep <code>core/</code> , <code>gui/</code> , <code>protocols/</code> , <code>stylesheet</code> , and <code>demo</code> beside <code>main.py</code>
Network	Local UDP/TCP sockets; isolated laboratory for external receivers
Optional tools	Wireshark/capture reader and an independently configured PDC

Windows supports the Qt interface and main UDP workflow. GOOSE uses Linux `AF_PACKET`; native Windows does not support that publisher. Linux/WSL GOOSE also depends on raw socket permissions and the interface. The core experiment does not require elevating the desktop application.

Windows installation

Open PowerShell in the directory containing `main.py`:

POWERSHELL

```
py -3 --version
py -3 -m venv .venv
.\venv\Scripts\python.exe -m pip install -r requirements.txt
.\venv\Scripts\python.exe -c "import PyQt6, numpy, matplotlib; print('Core imports OK')"
.\venv\Scripts\python.exe main.py --demo
```

Calling the interpreter directly avoids a PowerShell activation-policy change. Use an existing working environment if you already have one. Do not reuse an environment copied from another OS or machine.

Linux or WSL installation

BASH

```
python3 --version
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -c "import PyQt6, numpy, matplotlib; print('Core imports OK')"
.venv/bin/python main.py --demo
```

If environment creation fails, install your distribution's Python virtual-environment package. Qt may also need XCB, OpenGL, EGL, font, and D-Bus libraries.

The bundled `install.sh` targets Ubuntu 22.04 and creates `venv/`. It invokes `sudo apt-get`, selects Python, installs system packages, and installs requirements. Review it for other distributions. Its system-package block suppresses failures with `|| true`; a success banner alone does not verify the installation.

BASH

```
bash install.sh
venv/bin/python main.py --demo
```

Display setup

On Windows, use a normal desktop session. Under WSL, use an available WSLg display or a configured display server. Preserve working `DISPLAY` and `WAYLAND_DISPLAY` values.

On Linux, `main.py` sets `QT_QPA_PLATFORM=offscreen` when neither display variable exists, unless a platform was already set. Offscreen mode supports checks but produces no visible window. It is not a headless simulation service.

First experiment

1. Launch with `--demo`; it loads a topology without starting simulation.
2. Choose **Scenarios > Clean Baseline**.
3. Click **Validate**; the demo has a PMU, Local PDC, and attached Threat Agent.
4. Click **Run** (F5). Check packet totals and proxy status.
5. Inspect magnitude, frequency, and angle. Clean values overlap, apart from possible startup/configuration-record artifacts.
6. Click **Stop** (F6).
7. Choose **Scenarios > Noisy Sensor**, then run again. Modified magnitude varies around the original.
8. Stop, save the topology, and export incident JSON/CSV from the Integrations export tab.

The demo forwards UDP to `127.0.0.1:4713`. A Local PDC node represents a destination; it does not launch a PDC receiver. The GUI can display proxy results with no listener. See [Integrations](#) for receipt verification.

Installation verification

Windows:

POWERSHELL

```
.\.venv\Scripts\python.exe test_regression.py
.\.venv\Scripts\python.exe docs/examples/inspect_frame.py
```

Linux/WSL:

BASH

```
.venv/bin/python test_regression.py
.venv/bin/python docs/examples/inspect_frame.py
```

The regression suite uses offscreen Qt. The documentation example verifies a deterministic transformation without network traffic. See [Testing](#) for results and coverage limits, then [User guide](#) to build an experiment.

Source: entry point (`main.py`), requirements (`requirements.txt`), installer (`install.sh`), demo (`mitm-demo-topology.json`).

CHAPTER 03

03 User guide

Workspace and editing

The left panel contains the hierarchical node palette and protocol selection. The center contains the topology canvas; the right panel contains Properties, Attacks, and Integrations. Lower panels show waveforms and packet records. The toolbar provides simulation, file, editing, and zoom controls.

1. Drag a palette node onto the canvas, double-click a palette item, or use a toolbar node shortcut.
2. Drag nodes to arrange them. Select a node for properties; right-click for its configuration menu.
3. Choose **Connect**, click source, then destination. Escape cancels an unfinished link.
4. Select or right-click a link to inspect/change its protocol.
5. Move a Threat Agent onto a link to record interception visually.
6. Use **Delete** mode or select items and press Delete/Backspace to remove them.
7. Save with Ctrl+S.

The canvas is a diagram editor. A drawn switch does not create packet-switching behavior, and every link does not launch a network connection.

Node palette

Level	Nodes	Current meaning
L0 Process	CT/VT Sensor, Circuit Breaker	Diagram assets with sensor/breaker metadata
L1 Bay	PMU, Protection IED, Bay Control Unit	PMU selects the stream; IED/BCU illustrate bay devices
L2 Station	Local PDC, Station Switch, Station HMI, Engineering Workstation, Gateway/RTU	Local PDC selects destination; other nodes retain station metadata
L3 State	State PDC	Regional diagram and uplink indicator
Unassigned	Threat Agent, Virtual Node	Interception annotation and generic device

Breaker/uplink toggles update configuration and visuals; they do not actuate physical devices or connect a regional PDC.

Link defaults are GOOSE for L0-L1; C37.118 for PMU links between L1-L2 and GOOSE for other L1-L2 links; DNP3 for Gateway/RTU links between L2-L3 and C37.118 for other L2-L3 links. Other combinations keep the selected canvas protocol. Explicit link choices override defaults.

Validate and run

Validate (Ctrl+Shift+V) blocks startup for missing PMU or Local PDC nodes. State PDC does not substitute for Local PDC. Missing direct C37.118 PMU-PDC links, unattached agents, and duplicate PMU ports produce warnings. Run asks whether to continue when warnings exist.

The first PMU sets the proxy listen port. The first Local PDC sets destination IP/port. Desktop runtime uses UDP, 30 frames/s, 50 Hz, and 120 V nominal magnitude regardless of several editable node fields.

Stop before loading another topology, changing endpoints, or selecting a scenario. Ordinary Open does not consistently enforce this, and a scenario can change attack state even when loading its demo is blocked during a run.

Guided scenarios

Scenario	Module	Preset
Clean Baseline	Disable current attack	No transformation
Noisy Sensor	NOISE	noise_std=5.0
GPS Frequency Spoofing	FREQUENCY_OVERRIDE	target_freq=60.0
Voltage False Data	MAGNITUDE_OVERRIDE	phasor_idx=0, value=0.0
Packet Loss Drill	DROP	drop_percent=50.0
Replay Drill	REPLAY	buffer_size=90

Each loads the bundled diagram. GPS Frequency Spoofing changes a measurement field; it does not simulate GPS hardware. Presets are not complete run checkpoints. Reset attack statistics/state and review Schedule before comparing experiments; stop/start does not automatically reset them.

Manual attacks

In **Attacks**, disable the current module, select the desired dropdown entry, adjust parameters, and enable it. Confirm the active module in the panel. Parameter edits are synchronized through the enable/disable action; changing the dropdown alone need not change the engine.

Use the dropdown as the authoritative selector. The top-level Attacks menu currently opens/synchronizes the panel without selecting its requested module. The selected-link scope option does not implement per-link execution.

Enable **Schedule attack by packet frame** to set Start after and Duration. Zero duration has no scheduled end. Read [Attack scheduling](#) for exact frame indexing.

Reset Stats clears engine counters and module state, including replay buffers and ramp/pulse counters. It does not clear integration incident history or remove the schedule.

Interpret results

Waveforms

Plots compare original and modified phase-A magnitude, frequency, and phase-A angle. Each retains 120 samples; the active tab refreshes at 10 Hz. At 30 data frames/s this is approximately four seconds.

The horizontal axis uses local processing time, not the frame timestamp. Dropped and configuration records can reach the plots. A plotted value does not prove delivery, and packet loss need not appear as a gap. Replay retains old frame timestamps but appears at the current display time.

Packet classifications

Status	Meaning
clean	Unchanged, filter bypass, inactive schedule, or no content modification
attacked	Engine marked content modified and proxy attempted re-encoding
dropped	Attack selected the frame for non-forwarding
invalid	Parser rejected or did not recognize the frame

Delay preserves bytes and remains `clean`. Dashboard counters count matching data frames passed to the engine; proxy and incident totals also include other records.

Exports

The Integrations export tab provides JSON and CSV. JSON retains original/modified frame fields; CSV summarizes events. STIX is experimental. PCAP needs an active writer and packet bytes; the current desktop capture path is incomplete. See [Integrations](#).

Keyboard reference

Action	Shortcut
New / Open / Save	Ctrl+N / Ctrl+O / Ctrl+S
Run / Stop	F5 / F6
Validate	Ctrl+Shift+V
Zoom in / out / fit	Ctrl+= / Ctrl+- / Ctrl+0
Fit with canvas focus	F
Pan	Middle-button drag or hold Space and drag
Zoom around pointer	Mouse wheel
Cancel link / select	Escape
Delete selection	Delete / Backspace
Exit	Ctrl+Q

Record an experiment

Record source revision, package versions, topology, actual endpoints, module parameters, schedule, reset procedure, duration, and counters. Export before closing. For receiver tests, record transport/configuration and independent receipt evidence. Noise/drop results vary because the GUI has no random-seed setting.

Source: main window (gui/main_window.py), canvas (gui/canvas.py), nodes (gui/node_types.py), attack panel (gui/attack_panel.py), viewer (gui/waveform_viewer.py).

CHAPTER 04

04 Architecture

System context

GridSec Sim is a local desktop application. It owns a synthetic PMU and a forwarding proxy; an external PDC, capture analyzer, or event collector is a separate system. No database, message broker, cloud backend, or web application is required.

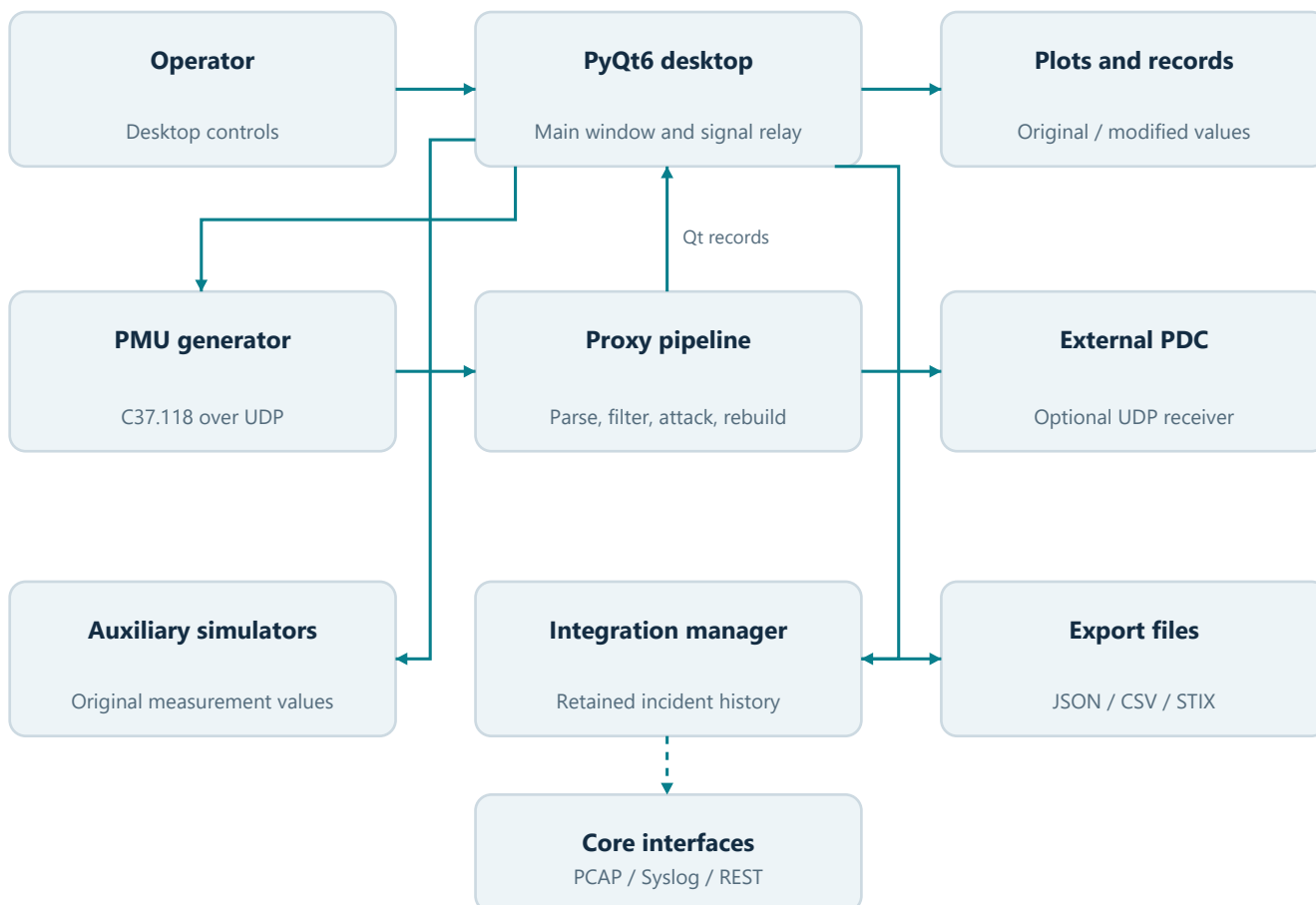


Figure 1. System context and information flow.

The dotted path identifies core integration capabilities whose desktop wiring is incomplete. Auxiliary protocols are updated from original measurements, not the modified C37.118 values.

Components and ownership

Component	Responsibility and owned state
main.py	Dependency check, logging, Qt application, crash handler, demo scheduling
MainWindow	Runtime component handles, shared engine/filter, UI composition, Qt relay, integration manager
TopologyCanvas	Node/link collections, IDs, attachments, drawing and persistence
PMUSimulator	UDP generation thread, its own codec, measurement controls, send counters
PDCProxy	Receive/forward sockets, processing counters, callbacks, engine/filter references
packet_parser	Module-global active codec used for parse/rebuild
AttackEngine	Ten module instances, selected type, counters, schedule
TrafficFilter	Rules controlling which data frames enter the attack engine
WaveformViewer	Rolling measurement buffers and plot timer
IntegrationManager	In-memory incidents/packet summaries, session ID, export and integration configuration
rest_api	HTTP handler and module-global callback/auth/SSE state

Startup and shutdown

1. `main.py` configures logging and checks PyQt6, matplotlib, and numpy imports.
2. A `QApplication` and `MainWindow` are constructed. Background simulation has not started yet.
3. `--demo` schedules loading of the bundled topology.
4. Run validates the diagram and selects its first PMU and Local PDC.
5. The UDP proxy starts; the GUI waits 150 ms before starting the PMU. This is a fixed delay, not a readiness handshake.
6. The PMU sends a configuration frame, then data at the configured constructor rate, fixed to 30 frames/s by the GUI.
7. The GUI starts auxiliary GOOSE, DNP3, and Modbus components and plotting.
8. Stop requests component shutdown and stops plot updates. Closing a running main window invokes that simulation stop path.

Threads are daemon-based and components generally signal stop/close sockets rather than synchronously join every worker. The main window does not explicitly stop the integration manager on close. Embedders should manage integration lifecycle themselves and avoid immediate port reuse until workers exit.

Proxy pipeline

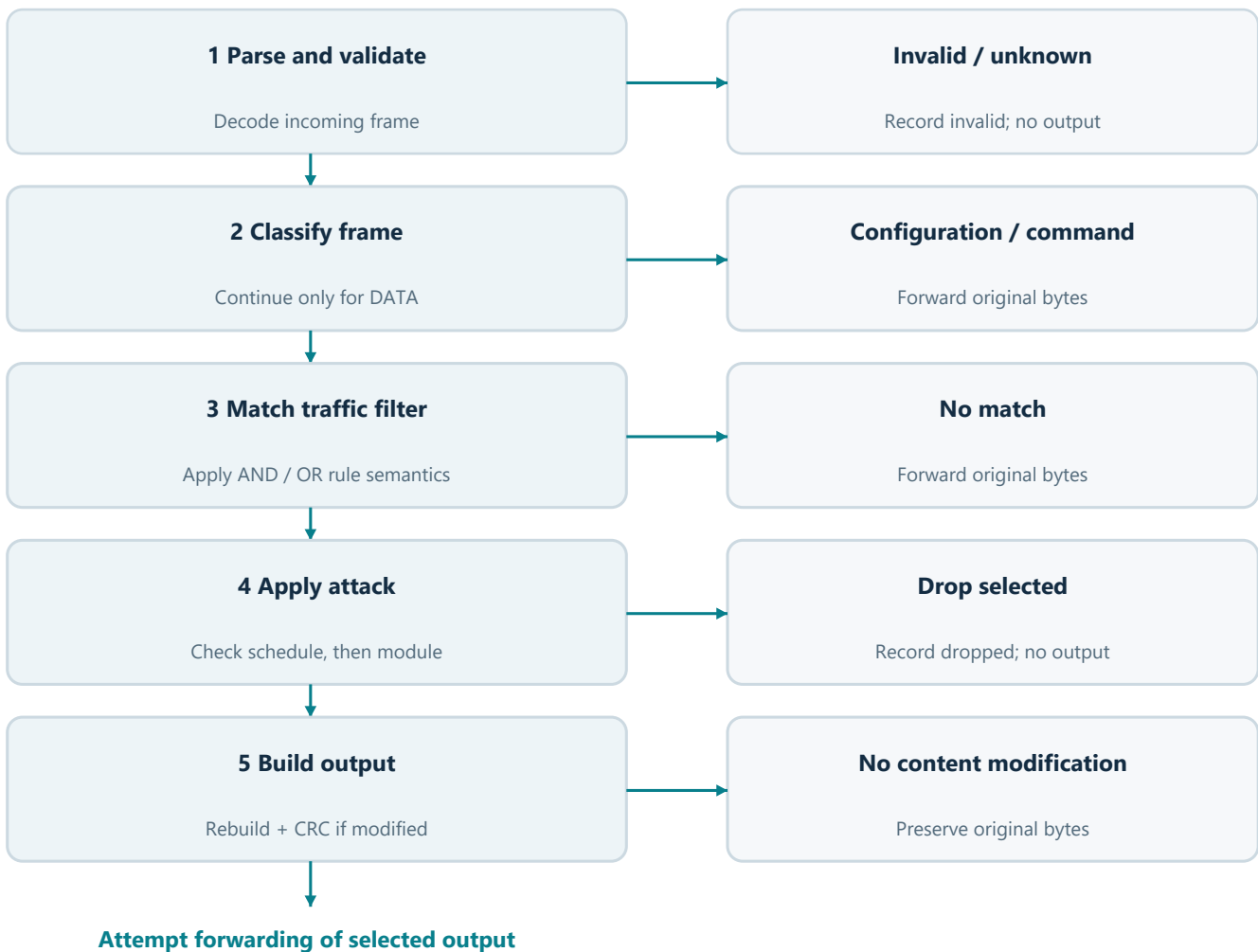


Figure 2. Proxy processing and disposition paths.

PacketRecord is emitted during processing, before UDP forwarding is attempted. Thus a successful record or plot update is not a delivery acknowledgment. Send failures increment proxy errors. Rebuild failures can return the original raw bytes while the record remains classified as attacked.

Configuration and command frames bypass attack/filter processing once decoded. Unknown and invalid types are discarded. Scheduling counts matching data frames entering the engine, not all received packets.

Threads and synchronization

PMU and proxy callbacks emit signals through `WorkerRelay`. Qt delivers connected UI work on the main thread. The GUI batches log updates every 200 ms, refreshes status every second, and updates attack statistics every 500 ms. The viewer redraws only its active tab at 10 Hz.

PMU measurement controls and integration record dequeues use locks. The engine is shared between UI and proxy without a comprehensive locking protocol. API callbacks execute on the server thread. Future remote control must marshal UI changes to Qt rather than manipulating widgets there.

The REST server runs `HTTPServer.serve_forever()` on a background thread. Request handling itself is single-threaded. An SSE connection occupies that handler until it ends and can prevent other requests from being served. Multiple server instances also share module-global state.

State and persistence

- Topology JSON stores diagram state; loading it does not recreate a complete running session.

- Engine state survives stop/start within one main window unless reset.
- Integration incidents survive simulation stops and rotate when their bounded deque fills.
- There is no automatic session recovery, database persistence, or persistent integration-settings file.
- Parser codec and REST state are global within the Python process, limiting safe multi-stream/multi-server composition.

The parser defaults to zero digital words while the GUI PMU produces one. Passing `codec=` to `PDCProxy` does not update the parser's global codec. See [Known limitations](#) and the explicit `set_codec()` call in the offline example.

Source: entry (`main.py`), main window (`gui/main_window.py`), proxy (`core/pdc_proxy.py`), parser (`core/packet_parser.py`), manager (`core/integration_manager.py`), REST (`core/rest_api.py`).

CHAPTER 05

05 Configuration reference

Launch options and environment

Setting	Default	Effect
python main.py	Empty canvas	Starts the desktop application
python main.py --demo	Opt-in	Loads bundled diagram after startup; does not start traffic
GRIDSEC_LOG_LEVEL	INFO	Python logging level, uppercased; unknown names fall back to INFO
DISPLAY, WAYLAND_DISPLAY	Environment-dependent	Linux/WSL display discovery
QT_QPA_PLATFORM	Qt/platform-dependent	Can select offscreen Qt for automated checks

There is no project CLI parser exposing endpoint, attack, API, capture, config-file, or headless-run flags. --demo is detected directly in sys.argv.

PowerShell logging example:

POWERSHELL

```
$env:GRIDSEC_LOG_LEVEL = 'DEBUG'  
.\.venv\Scripts\python.exe main.py --demo
```

Linux example:

BASH

```
GRIDSEC_LOG_LEVEL=DEBUG .venv/bin/python main.py --demo
```

Dependency declaration

Package	Declared minimum	Role
PyQt6	6.4.0	Desktop UI and signals
matplotlib	3.7.0	Embedded waveform plots
numpy	1.24.0	Plot data arrays
scapy	2.5.0	Declared packet-tooling dependency; core PCAP writer uses standard-library code
pyqtgraph	0.13.0	Declared dependency; current waveform implementation uses matplotlib
cryptography	41.0.0	Declared dependency; its presence does not enable TLS/authentication on protocol streams

No exact lockfile or package metadata is supplied. Record installed versions for repeatable experiments. Startup explicitly checks only PyQt6, matplotlib, and numpy.

Desktop runtime settings

Setting	Actual runtime value/source	Notes
PMU destination host	127.0.0.1	Editable PMU IP is not used as the socket destination
Proxy bind host	127.0.0.1	Fixed by main window
Proxy listen port	First PMU node's port; default 4712	Stop and restart after edits
Proxy destination	First Local PDC node's ip and port; defaults 127.0.0.1:4713	Represents an independently running receiver
Main transport	UDP	Saved node proto=TCP does not switch the desktop runtime
Reporting rate	30 frames/s	PMU node reporting_rate is not passed through
PMU ID	1	Node idcode is not passed through
Measurements	Three phasors, one analog, one digital word at generation	Parser digital-word mismatch is documented separately
Nominal frequency/magnitude	50 Hz / 120 V	Constructor arguments fixed by main window
GOOSE	App ID 0x0001, IED name GridSecSim, 1 Hz, autodetected interface	Started independently of diagram node counts
DNP3	Destination 127.0.0.1:20000, requested bind port 20000, 1 Hz	Bind host is 0.0.0.0; may fall back to an ephemeral port
Modbus	Server 127.0.0.1:502, 1 Hz master polling	On bind failure attempts port 10502

Saved properties versus live behavior

Node labels, positions, levels, protocol labels, breaker state, switch port count, PDC buffer settings, regional uplink state, and Threat Agent metadata are stored in the diagram. Only the runtime fields identified above determine the desktop sockets.

Editing a property while running does not rebuild active components. `_on_config_applied()` logs the change; it does not reconfigure the proxy. Stop, edit, validate, and restart.

Threat Agent attack type, parameters, and schedule may be recorded when toggling attacks through the panel. Loading them restores metadata, not engine execution. The one active engine remains global to the stream; selected-link scope is not enforced.

Core constructors

Interface	Useful defaults and controls
PMUSimulator	dst_host='127.0.0.1', dst_port=4712, idcode=1, fps=30 clamped to 1-120, nom_freq=50.0, nom_voltage=120.0
PDCProxy	listen_host='0.0.0.0', listen_port=4712, target_host='127.0.0.1', target_port=4713, proto='UDP'; core also has TCP mode
C37118Codec	idcode=1, three phasors, one analog, zero digital words unless explicitly set
GridSecRESTServer	host='127.0.0.1', port=8080; bearer auth enabled in shared API state
IntegrationManager.configure_rest_api	Defaults to host='0.0.0.0', port=8080; explicitly pass loopback for local use
configure_syslog	host='127.0.0.1', port=514, protocol='UDP'
configure_pcap	/tmp/gridsec_capture.pcap, regular file by default; choose a native path on Windows

Constructors expose more flexibility than the desktop. These Python interfaces are not a promise of complete interoperability or thread-safe multi-instance use.

Persistence and reset

State	Saved/recovered?
Topology nodes, links, attachment IDs	Explicit JSON save/load
Attack metadata on agents	May be saved; not automatically armed on load
Engine counters, ramp/replay state	Memory only; retained across stop/start until reset
Incident history	Memory only, last 10,000 records
Recent packet summaries	Memory only, last 1,000 records
API token	Generated in module state per process; explicit setter available
Integration host/path settings	Memory only; no automatic settings file
Logs	Console by default; file persistence requires explicit redirection

Source: requirements (requirements.txt), entry (main.py), main window (gui/main_window.py), PMU (core/pmu_simulator.py), manager (core/integration_manager.py).

CHAPTER 06

06 Attack engine reference

Execution contract

AttackEngine holds one instance of each module and selects one active type. `set_attack(AttackType, params)` disables the previous module and enables the selected one. Module parameters and internal state generally persist between selections. `NONE` is the default passthrough state.

`apply(frame_dict)` returns `(frame_dict, was_modified, was_dropped)`. Modules operate on decoded C37.118 data frames. They do not intercept GOOSE/DNP3/Modbus traffic, transmit exploit payloads to unrelated devices, or implement packet-routing policies.

Phasor indices are zero-based: 0 is phase A in the default generator. Magnitudes use the generator's voltage units, frequency is Hz, angles in decoded frames are radians, and the angle-override parameter is degrees.

Parameters

These are **engine parameter names**, accepted by `set_params()`. Unknown keys are generally ignored, not rejected. UI ranges may be narrower than core setters.

Type	Parameters and defaults	Behavior and core constraints
NOISE	<code>noise_std=1.0</code> , <code>add_freq_noise=false</code> , <code>freq_std=0.05</code>	Independent Gaussian noise on all magnitudes; optional frequency noise. Uses <code>noise_std</code> , not <code>std_dev</code> . Magnitudes are not clamped to zero.
RAMP	<code>ramp_rate=0.05</code> , <code>target='magnitude'</code> , <code>direction='up'</code> , <code>phasor_idx=-1</code>	Adds/subtracts <code>rate × module frame count</code> . Target: magnitude/frequency; negative index selects all phasors. Magnitudes are clamped at zero.
PULSE	<code>amplitude=30.0</code> , <code>interval=30</code> , <code>duration_frames=3</code> , <code>target='magnitude'</code> , <code>phasor_idx=-1</code>	Spike when <code>frame_count % interval < duration_frames</code> . Interval/duration are clamped to at least 1. Amplitude units follow target.
FREQUENCY_OVERRIDE	<code>target_freq=60.0</code>	Replaces <code>FREQ</code> ; setter clamps to 45-65 Hz.
MAGNITUDE_OVERRIDE	<code>phasor_idx=0</code> , <code>value=0.0</code>	Replaces one magnitude; out-of-range index passes unchanged.
ANGLE_OVERRIDE	<code>phasor_idx=0</code> , <code>angle_deg=0.0</code>	Replaces one angle after degree-to-radian conversion; invalid index passes unchanged.
REPLAY	<code>buffer_size=30</code>	Records N eligible frames unchanged, then loops deep copies including old timestamps. Buffer size is at least 1. recording and buffered are readback fields, not writable parameters.
DELAY	<code>delay_ms=100.0</code>	Sleeps on the proxy thread for each eligible data frame. Setter clamps to 0-5000 ms. No content change/drop flag.
DROP	<code>drop_percent=20.0</code>	Independent random drop decision; setter clamps to 0-100%.
SCALE	<code>scale_factor=1.5</code> , <code>also_scale_freq=false</code>	Multiplies all magnitudes and optionally frequency. Core setter does not enforce the 0-10 range mentioned in its docstring.

Timing and state

Ramp is measured per module invocation, not per wall-clock second. Pulse starts with module frame count 1, so its first window is not a zero-based sequence. A duration at least as large as its interval makes every eligible invocation a pulse.

Replay requires N frames to fill before modifications begin. At an uninterrupted 30 eligible frames/s, a 90-frame buffer takes approximately three seconds. Restarting the simulation does not automatically clear the engine's buffer. Applying a new `buffer_size` recreates it; `reset_attack_state()` clears all module state.

Delay reduces proxy throughput because it blocks that stream's processing worker. The PMU may continue generating faster than processing, creating socket backlog or loss. Delay does not increment `modified_packets` or produce attacked incidents by itself.

Scheduling

PYTHON

```
engine.set_attack(AttackType.SCALE, {"scale_factor": 2.0})
engine.reset_stats()
engine.reset_attack_state()
engine.set_schedule(start_frame=2, duration_frames=2)
```

The engine increments `total_packets` before checking the window. For a positive duration, the active condition is:

TEXT

```
start_frame <= total_packets < start_frame + duration_frames
```

A duration of zero removes the upper bound. `start_frame=0, duration_frames=0` is immediate and indefinite. With start 2 and duration 2, calls 1-4 produce modification flags `[false, true, true, false]`. To get N complete clean eligible frames first, use start N+1.

Counters are cumulative since the last statistics reset. Arming a schedule during a run uses that existing counter, not a delay relative to the moment you clicked Enable. Disabled/schedule-inactive behavior and module-specific counters also mean a ramp's local frame count is distinct from the engine counter. Use both resets for a controlled repeat.

Start/stop does not reset scheduling. The GUI may show "Waiting" after a finite window has ended. A selected attack shown in the dashboard can be enabled but outside its active window.

Traffic filtering

`TrafficFilter` decides whether data enters the engine; nonmatching data is still forwarded unchanged. It is not a firewall.

PYTHON

```
from core.traffic_filter import TrafficFilter

traffic_filter = TrafficFilter()
traffic_filter.add_rule(dst_ip="127.0.0.1", dst_port=4713, proto="UDP")
```

Rule fields are `src_ip`, `dst_ip`, `src_port`, `dst_port`, `proto`, `label`, and `enabled`. None is a wildcard. Specified fields combine with AND; enabled rules combine with OR. Empty rule list matches everything; a nonempty list containing only disabled rules matches nothing. IP matching is string equality, not CIDR/subnet matching.

In UDP proxy metadata, destination means the proxy's **forwarding destination**, not its listen address. The generated PMU source port is an ephemeral socket port. The desktop constructs an empty filter and does not expose a working rule editor or connect selected-link scope to these rules.

Counters and interpretation

`total_packets` counts engine calls. `modified_packets` increments when a result is modified; `dropped_packets` increments when dropped. Non-data frames and filter bypasses do not enter the engine. Reset Stats clears these and module state through the GUI, but does not clear incident history.

A modification flag expresses module behavior, not necessarily a numerical difference: overriding a value with the same value can still count as modified. Rebuild failures and downstream send errors require separate inspection.

Use [inspect_frame.py](#) for a deterministic example. Source: engine (`core/attack_engine.py`), filter (`core/traffic_filter.py`), proxy (`core/pdc_proxy.py`), panel (`gui/attack_panel.py`).

CHAPTER 07

07 Protocol reference

This reference describes repository behavior. Protocol names identify the intended wire formats; support here is not a statement of full standards conformance or vendor interoperability.

Support matrix

Protocol	Codec/helpers	Live implementation	Main attack path
C37.118	Data, CFG-2, command encoding/decoding, CRC	UDP PMU + UDP proxy in GUI; TCP proxy mode in core	Yes, data frames
GOOSE	Ethernet framing, BER dataset helpers and decoding	Linux raw Ethernet publisher	No
DNP3	Link framing/CRC, transport/app helpers, analog/binary objects	UDP outstation-style simulator	No
Modbus TCP	MBAP/PDU helpers, register conversion and decoding	TCP server and polling master	No
IEC104	APDU and I/U-frame helpers, frame decoding	No live desktop simulator	No

Auxiliary simulators start once per desktop run, independently of how many protocol-specific nodes exist. They receive original PMU values. A red C37.118 waveform does not establish modified values on these other protocols.

C37.118 implementation

C37118Codec uses big-endian fields and float32 polar phasors. Default data representation is magnitude/angle pairs, float32 frequency and ROCOF, float32 analog channels, and unsigned 16-bit digital words. Station configuration is constructor-based, not dynamically negotiated for arbitrary incoming devices.

Generated data frame

Field	Bytes
SYNC	2
FRAMESIZE	2
IDCODE	2
SOC	4
FRACSEC	4
STAT	2
Phasors	8 × phasor count
FREQ	4
DFREQ	4
Analog	4 × analog count
Digital	2 × digital-word count
CRC	2

Total encoded size is $26 + 8P + 4A + 2D$ bytes. The GUI generator's three phasors, one analog, and one digital word produce **56-byte data frames**. Use encoded byte length/FRAMESIZE when checking this; the codec's private `_data_frame_size` estimate omits STAT.

The PMU sends a CFG-2 frame once at startup, then generates data at 30 frames/s in the GUI. Its model includes slow frequency variation around 50 Hz and small voltage variations around 120 V, with three nominal phase angles. It is synthetic telemetry, not an electrical solver.

Data decoding checks CRC and structural availability using configured channel counts. It does not adapt its layout from a received CFG-2 frame. The parser recognizes configuration variants, but the implementation's main configuration encoder/decoder is CFG-2-oriented. Command helpers do not establish a complete bidirectional PMU/PDC command session.

The parser's active codec defaults to zero digital words, while the PMU creates one. `PDCProxy(codec=...)` stores the codec without configuring the global parser. For embedded experiments, explicitly call `packet_parser.set_codec()` with matching counts before parsing/rebuilding. The unmodified path preserves original bytes; the modified desktop path can lose the digital word.

TCP boundary

The core proxy can accept TCP and extract length-delimited frames before forwarding on a TCP connection. The GUI always creates UDP components, and its PMU generator is UDP-only. Changing a node's Transport field to TCP does not establish a TCP test. Reverse command forwarding and compatibility with a receiver's command handshake must not be assumed.

GOOSE

The codec builds Ethernet frames with EtherType `0x88B8`, a GOOSE application header, and BER-encoded values. The desktop publisher uses App ID `0x0001` and a dataset ordered as Va, Vb, Vc, frequency, trip, breaker-closed. It includes sequence/state counters and a 2000 ms time-allowed-to-live setting.

Live transmission uses Linux AF_PACKET and requires an appropriate raw-socket permission/interface. It is unavailable in native Windows; startup can produce a worker-thread exception for the missing socket family. GOOSE is Layer 2 traffic: the IED node's displayed port 102 is metadata, not a GOOSE TCP port.

DNP3

DNP3DataLink encodes link frames and inserts/verifies header and 16-byte-block CRCs. Higher-level helpers build transport/application payloads, floating analog inputs, binary inputs, read requests, and unsolicited responses.

The desktop simulator requests UDP bind 0.0.0.0:20000 and sends to 127.0.0.1:20000 at 1 Hz. It falls back to an ephemeral local bind port if binding fails. It maintains sender/listener workers and handles supported read requests. The source outstation address is 1.

Analog measurements are Va, Vb, Vc, frequency, active power, and reactive power; binaries are trip and breaker-closed. This is a limited simulator, not a complete outstation implementing all object groups, authentication, or production session behavior.

Modbus TCP

The implementation provides a TCP register server and local polling master. The server handles read input registers (function 04), read holding registers (03), and write single register (06), returning supported exception responses for other cases. It has ten input registers and sixteen holding registers.

Input addresses are zero-based PDU offsets:

Offset	Value	Decode register by dividing by
0	Va	10
1	Vb	10
2	Vc	10
3	Frequency	100
4	Active power	1
5	Reactive power	1
6	Ia	10
7	Ib	10
8	Ic	10
9	Power factor	1000

Values are rounded and clamped to unsigned 16-bit range. They are scaled integers, not pairs of registers containing IEEE float32. GUI defaults are 127.0.0.1:502, polling at 1 Hz. Bind failure attempts port requested_port + 10000, normally 10502; inspect actual status/logs.

IEC104

IEC104Frame and helpers support APDU representation, STARTDT/TESTFR U-frames, I-frame construction, and frame decoding. There is no desktop IEC104 socket worker, station model, or attack pipeline. The canvas label is available for diagramming.

Source: C37.118 (protocols/c37118.py), GOOSE (protocols/goose.py), DNP3 (protocols/dnp3.py), Modbus (protocols/modbus.py), IEC104 (protocols/iec104.py), simulator wiring (gui/main_window.py).

CHAPTER 08

08 REST API reference

Availability and base URL

The HTTP implementation is `core/rest_api.py`, using Python's standard-library `HTTPServer`. It can be started through the core `GridSecRESTServer` or `IntegrationManager`. The desktop REST Start button currently calls a missing convenience method, so use the documented core example to explore the API.

TEXT

```
http://127.0.0.1:8080/api/v1
```

The documentation example uses port **18080** to distinguish it from application defaults:

POWERSHELL

```
.\.venv\Scripts\python.exe docs/examples/serve_api.py
```

It serves one synthetic incident without starting protocol simulators or enabling remote attacks. Copy the bearer token printed locally. Stop with `Ctrl+C`. Do not run multiple API examples in the same Python process: state is module-global.

Authentication

All implemented GET routes except `/api/v1/status` and all POST routes require:

HTTP

```
Authorization: Bearer YOUR_LOCAL_API_TOKEN
```

Initial state enables authentication and generates a token with `secrets.token_hex(16)`. `set_api_key()` changes it; `disable_auth()` and `enable_auth()` control enforcement. Setting an empty token does **not** itself disable auth, despite older comments suggesting otherwise.

The token is shared across server instances and logged at INFO when starting the server. There is no TLS, account model, role separation, expiry, rate limiting, or durable audit log. Bind loopback for local work and protect token-bearing logs. See [Security](#).

Routes

Method and path	Input	Response
GET /api/v1/status	None; public	Tool/version/time/auth plus integration status
GET /api/v1/simulation	None	Status callback result without the tool envelope
GET /api/v1/topology	None	Latest topology callback snapshot, or {}
GET /api/v1/attacks/current	None	Attack callback result, or {}
GET /api/v1/incidents	limit default 100; optional proto	{count, total, incidents}
GET /api/v1/packets/recent	limit default 50	{count, packets}
GET /api/v1/export/stix	None	JSON attachment gridsec_stix2.json
GET /api/v1/export/csv	None	CSV attachment gridsec_incidents.csv
GET /api/v1/export/pcap	None	PCAP attachment gridsec_capture.pcap, or 404
GET /api/v1/events	None	Server-Sent Events stream
GET /api/v1/docs	None	Authenticated embedded HTML help
POST /api/v1/attack/enable	JSON type and optional params	{success, attack_type}
POST /api/v1/attack/disable	Empty object accepted	{success: true, message}
OPTIONS	Preflight	204; GET/POST/OPTIONS and Authorization/Content-Type permitted

Paths have trailing slashes removed before dispatch. / becomes an empty string, so the attempted root documentation route does not resolve; use /api/v1/docs. There are no HTTP routes to start/stop simulation, upload topology, create incidents, set schedules, or ingest Splunk HEC data. Comments mentioning those integrations do not add endpoints.

Status response

Illustrative manager-backed response:

JSON

```
{
  "tool": "GridSec Sim",
  "version": "1.0.0",
  "timestamp": 1788825600.0,
  "auth": true,
  "sim_running": false,
  "session_id": "11111111-1111-4111-8111-111111111111",
  "uptime_s": 12,
  "total_incidents": 1,
  "attacked": 1,
  "integrations": {"pcap": false, "syslog": false, "rest_api": true}
}
```

uptime_s is time since manager construction. total_incidents and attacked count the retained history, not lifetime totals. rest_api=true tests manager handle presence rather than an independent health probe. In the desktop, sim_running can remain stale after Stop because packet callbacks set it true without resetting it there.

/attacks/current currently receives engine statistics from the desktop (total_packets, modified_packets, dropped_packets), despite its name suggesting type/parameters. A custom embedding can return a different object.

Incident and packet queries

POWERSHELL

```
$taskHeaders = @{ Authorization = 'Bearer YOUR_LOCAL_API_TOKEN' }  
Invoke-RestMethod 'http://127.0.0.1:18080/api/v1/status'  
Invoke-RestMethod 'http://127.0.0.1:18080/api/v1/incidents?limit=10' -Headers $taskHeaders  
Invoke-RestMethod 'http://127.0.0.1:18080/api/v1/packets/recent?limit=10' -Headers $taskHeaders
```

Linux equivalent:

BASH

```
curl -H 'Authorization: Bearer YOUR_LOCAL_API_TOKEN' \  
  'http://127.0.0.1:18080/api/v1/incidents?limit=10'
```

Results preserve chronological order within the last N retained entries. limit has no validated upper/lower bound: limit=0 returns the entire retained list due to Python slicing, negative values have slice semantics, and nonintegers generally yield 500. Use positive integers.

The incident proto filter has a known mismatch: it looks for proto, while manager incident records use protocol. With the standard manager, a nonempty protocol filter returns no matches. Fetch without it and filter incident.protocol client-side.

See [Data formats](#) for incident and packet field dictionaries. The manager retains 10,000 incidents and 1,000 packet summaries. There is no cursor, durable sequence number, or guaranteed complete export after rotation.

Remote attack endpoints

Request shape:

JSON

```
{"type": "NOISE", "params": {"noise_std": 5.0}}
```

The correct parameter is noise_std; older embedded help uses the ineffective key std_dev.

IntegrationManager.set_attack_hooks(enable_fn, disable_fn) must connect the request to an engine. The desktop does not install these hooks. Without a hook, enable returns success=false; disable can return success while changing no engine state. A 200 response is not sufficient confirmation that a remote attack took effect.

Supported engine type strings appear in [Attacks](#). The HTTP layer itself does not validate enum membership, parameter ranges, or schedule fields. Custom hooks must validate data, manage concurrency, and report actual outcome. The example server deliberately leaves hooks unset.

SSE

BASH

```
curl --no-buffer -H 'Authorization: Bearer YOUR_LOCAL_API_TOKEN' \  
  'http://127.0.0.1:18080/api/v1/events'
```

The stream starts with a comment, sends : keepalive comments after about 30 idle seconds, and can emit:

TEXT

```
event: incident
data: {"status":"attacked","attack_type":"NOISE"}
```

Actual event data is a full incident object. The manager broadcasts only newly recorded attacked/dropped incidents. There is no initial replay, event ID, or Last-Event-ID support. Each subscriber queue has a capacity of 100; full queues are removed from broadcast delivery.

Current concurrency limit: HTTPServer processes requests serially. A live SSE connection can block other API requests and delay shutdown. Use polling for routine access and disconnect SSE clients before stopping the server. The fixed example incident was recorded before clients connect, so it will not arrive retrospectively via SSE.

Errors and response behavior

Code	Observed meaning
200	Handler completed; inspect success for attack commands
204	OPTIONS response
400	Missing attack type
401	Missing or incorrect bearer token
404	Unknown path or unavailable PCAP
500	Exception in GET handler, including invalid numeric limit

Unauthorized JSON contains error and an authorization hint. JSON responses use UTF-8 and permissive CORS *. Malformed JSON is converted to {} by POST handling. Other malformed POST structures or hook exceptions are not consistently converted to structured errors. Clients should tolerate a connection failure as well as JSON error responses.

Source: HTTP handler/server (core/rest_api.py), manager callbacks (core/integration_manager.py), desktop state wiring (gui/main_window.py).

CHAPTER 09

09 Integrations and exports

Integration status

Path	Current status
Main proxy > UDP PDC destination	Desktop-wired; external receiver setup and interoperability remain your responsibility
Incident JSON/CSV	Desktop export methods exist and receive proxy records
STIX	Exporter exists; experimental representation and taxonomy mappings
PCAP writer	Core implementation; GUI start/stop methods and raw-byte delivery are incomplete
Syslog/CEF	Core sender; GUI convenience-method wiring is incomplete
REST	Core server; GUI convenience methods and remote engine hooks are incomplete
Named vendor integrations	Generic format/interface possibilities, not tested product connectors

Connect a PDC receiver

The desktop sends UDP. Configure your laboratory receiver for the same transport and the generated C37.118 layout: ID 1, three float polar phasors, one float analog, one digital word, 50 Hz nominal, 30 frames/s. Match the receiver to the generator's CFG-2 and data layout rather than assuming every receiver can infer it automatically.

1. Start the receiver on a chosen lab address/UDP port.
2. Stop GridSec simulation, select the first **Local PDC**, and set its IP/port to that receiver.
3. Keep the first PMU's listen port distinct from the receiver port when both use loopback.
4. Validate and start a clean baseline.
5. Confirm incoming datagrams and accepted measurements at the receiver.
6. Apply a controlled transformation and verify the receiver's values, timestamps, and invalid-frame indicators.

openPDC is an intended receiver use case, but this review did not test an openPDC installation. A TCP-only receiver will not receive the GUI's UDP stream. The core TCP proxy does not turn the UDP generator into a complete command-driven PMU.

CFG-2 is sent once at PMU startup. If the receiver missed it, start the receiver first and restart the experiment. For modified data, account for the current parser digital-word mismatch described in [Protocols](#).

PCAP and Wireshark

PcapWriter creates classic Ethernet-linktype PCAP files with microsecond timestamps. UDP/TCP methods synthesize Ethernet/IP/transport headers around supplied application bytes. GOOSE accepts a supplied raw Ethernet frame. This is generated capture material, not a passive copy of actual NIC traffic or a complete TCP handshake.

Regular-file mode opens the destination with `wb`, replacing existing content. FIFO mode is POSIX-only, replaces an existing path with a named pipe, and waits for a reader in a background thread. Use a new dedicated output path. Check `is_active` and `error`; construction/start messages do not guarantee successful capture.

Current desktop limitations are two independent issues:

1. Capture buttons call `start_pcap_only()/stop_pcap_only()`, absent from `IntegrationManager`.

2. The main window retrieves `record.raw_bytes`, but `PacketRecord.__slots__` has no such field. The resulting empty bytes skip PCAP writes and appear as packet length zero.

The [offline frame example](#) can create a real sample PCAP through the core writer:

POWERSHELL

```
.\.venv\Scripts\python.exe docs/examples/inspect_frame.py --pcap sample-frame.pcap
```

Open the file in Wireshark. Its addresses are synthetic documentation values. For a genuine live-network trace, capture the relevant loopback/lab interface independently and compare it with the application records. The GUI's Wireshark launcher can find a native executable or attempt WSL on Windows, but a successful launcher does not fix capture wiring.

Syslog/CEF

The core sender formats a CEF message inside an RFC-5424-style syslog envelope. UDP is default; TCP sends newline-terminated messages. Default destination is `127.0.0.1:514`, facility `LOCAL7`.

When configured and started programmatically, the manager sends attacked, dropped, and invalid records; it does not send clean records. Severity is derived from status: clean `INFO`, attacked `HIGH`, dropped `MEDIUM`, invalid `LOW`. CEF numeric levels are 0, 7, 5, and 3 respectively.

For an authorized collector integration, use `configure_syslog()` and `start()` on the core manager, then call `record_packet()` as records arrive. Check collector receipt independently: UDP socket creation and `sendto()` success do not acknowledge ingestion. The sender provides no TLS, durable buffering, retry queue, or complete escaping of arbitrary user-controlled CEF fields.

The GUI's Syslog Test/Enable/Disable buttons call missing manager convenience methods. `patch_mgr.py` is an old source-rewriting helper, not a supported installation or runtime step.

JSON and CSV

JSON export includes session ID, export time, and retained incidents with original/modified fields. CSV contains nine event-summary columns. Both include clean records because the manager creates an Incident for every callback record, not only attacks.

Export before memory rotation or application exit. With roughly 30 records/s, 10,000 incident entries represent about 5.6 minutes, depending on configuration records and actual processing. These are retained records, not a durable experiment archive.

STIX

The exporter labels output as STIX 2.1 and builds identity, campaign, attack-pattern, indicator, course-of-action, relationship, and observed-data objects. Observed-data samples only the last 20 attacked incidents; it does not reproduce the full history.

The output uses custom ICS fields and static ATT&CK mappings. It has not been validated against a strict STIX validator or an external platform during this review. Broad port indicators do not independently identify an attack, and repository technique mappings require review before use as authoritative classifications. Retain JSON/CSV as the primary experiment record.

Core lifecycle example

PYTHON

```
from core.integration_manager import IntegrationManager

manager = IntegrationManager()
manager.configure_rest_api(host="127.0.0.1", port=18080)
results = manager.start()
try:
    # Your application supplies records through manager.record_packet(...).
    # Keep this process alive while clients use the server.
    print(results)
finally:
    manager.stop()
```

This fragment demonstrates lifecycle only; it exits immediately without a host event loop. Use [serve_api.py](#) for a runnable version. Start configured integrations once, inspect results, and stop them explicitly. Disconnect SSE clients before shutdown.

Source: manager/exporters (core/integration_manager.py), PCAP writer (core/pcap_writer.py), integration panel (gui/integration_panel.py), proxy (core/pdc_proxy.py).

CHAPTER 10

10 Data formats

Topology JSON version 2

New saves contain:

JSON

```
{
  "format": "GridSecSim",
  "version": 2,
  "nodes": [],
  "links": []
}
```

See the [complete example](#) and [JSON Schema](#). The schema describes portable v2 documents for tooling; the application loader does not validate against it or enforce format/version.

Nodes

Field	Type	Meaning
id	String	Stable topology identifier; new nodes use UUID strings; legacy IDs may become numeric strings
node_type	String	Registered node type
label	String	Display name
ip	String	Configured endpoint/diagram IP
port	Integer	Port metadata or runtime endpoint, depending on node
proto	String	Saved transport, normally UDP/TCP
level	Integer	Diagram level -1 through 3
x, y	Number	Scene coordinates
Other fields	Varies	Type-specific configuration

Registered type strings: CT_VT, BREAKER, PMU, PROTECTION_IED, BCU, PDC, SWITCH, STATION_HMI, ENGINEERING_WS, GATEWAY_RTU, STATE_PDC, THREAT_AGENT, VIRTUAL.

Examples of additional fields are PMU reporting_rate/idcode, PDC buffer_timeout/pmu_input_count, breaker state, switch port_count, engineering workstation high_risk, gateway protocol, and State PDC regional_uplink_connected.

Threat Agents may contain active, attack_type, attack_params, and attack_schedule (start_frame, duration_frames). intercepted_link is an in-process object ID that can appear in metadata; treat the link's threat_agent_ids as the portable relationship. running/active are visual state snapshots, not instructions to launch a process on load.

Links

Field	Type	Meaning
src_id, dst_id	String	References to node IDs
protocol	String	C37.118, DNP3, Modbus, IEC104, or GOOSE
threat_agent_ids	Array of strings	Attached Threat Agent node IDs

IDs must be unique, endpoints must exist, and attachment references should identify Threat Agents. JSON Schema cannot by itself enforce those cross-references; a semantic check is also required.

Legacy compatibility

Legacy files may omit format/version and identify nodes with `_id`, often integers. The loader accepts `id` or `_id`, maps both original and string representations, restores resolvable links, then restores Threat Agent attachments. The bundled demo is a legacy-shaped file with attachment references; saving through the canvas converts it to v2.

Missing endpoints cause a link to be skipped. Missing attachment references are ignored. Duplicate IDs can redirect references to the last mapped node. Loading clears the existing canvas first, so malformed input can leave an empty or partial diagram. Save current work before opening unfamiliar files.

Decoded C37.118 data

`parse_frame()` returns a dict or None. A decoded DATA dictionary includes:

Field	Representation
frame_type	data
sync, framesize, idcode	Integer header values
soc, fracsec	Integer wire timestamp fields
stat	Integer status word
phasors	List of magnitude/angle pairs; angles in radians
freq, dfreq	Hz and Hz/s
analog	List of float values
digital	List of unsigned 16-bit words
crc	Parsed CRC integer
raw	Original Python bytes

Other parser classifications include `cfg2`, `command`, `data_invalid`, `cfg_invalid`, `cmd_invalid`, and `unknown`; configuration detection also recognizes `cfg1/cfg3`. `rebuild_frame()` encodes data using the active codec and otherwise returns the stored raw bytes.

PacketRecord

The proxy's internal record has exactly these slots: `timestamp`, `original`, `modified`, `status`, `attack_type`, `description`, `src_addr`.

`timestamp` is local wall time when proxy processing begins; `src_addr` is the received socket address. There is no `raw_bytes` slot. Raw data can exist inside the frame dictionary's `raw`, but the current GUI does not use that as capture

input. Modified dictionaries can still contain original raw bytes, so blindly using that field would not provide the rebuilt wire output.

Incident format

`Incident.as_dict()` exports:

Field	Type/meaning
<code>id</code>	Eight-character UUID-derived string; not a durable globally unique event cursor
<code>timestamp</code>	Epoch seconds as float
<code>timestamp_iso</code>	UTC ISO-formatted timestamp
<code>protocol</code>	Protocol label, e.g. C37.118
<code>src_ip, dst_ip</code>	Recorded addresses
<code>attack_type</code>	Module name or NONE for unclassified clean records
<code>status</code>	clean / attacked / dropped / invalid
<code>severity</code>	INFO / LOW / MEDIUM / HIGH; CRITICAL exists as a model value
<code>description</code>	Generated event description
<code>original_val, modified_val</code>	Serialized frame dictionaries

The desktop currently supplies synthetic addresses `10.0.0.1` and `10.0.0.2`, source port 49000, destination port 4712, and protocol C37.118 to the integration manager, regardless of actual sockets. Do not use these fields as network-capture evidence.

Frame serialization stringifies values such as Python bytes; it does not define a base64 binary contract. Use actual PCAP bytes for packet reconstruction.

Export containers

JSON export has `session_id`, `exported_at`, and `incidents`. CSV column order is:

TEXT

```
id,timestamp_iso,protocol,src_ip,dst_ip,attack_type,status,severity,description
```

Recent packet API entries contain `timestamp`, `protocol`, `status`, `attack`, `src`, `dst`, and `len`. `src/dst` are address-and-port strings; `len` is the length of supplied raw bytes, currently zero for the GUI path.

The REST incident response adds `count` and `total`; packet response adds `count`. Exported STIX and PCAP semantics are described in [Integrations](#).

Source: canvas persistence (`gui/canvas.py`), node configurations (`gui/node_types.py`), parser (`core/packet_parser.py`), `PacketRecord` (`core/pdc_proxy.py`), incidents (`core/integration_manager.py`).

CHAPTER 11

11 Operations and troubleshooting

Deployment profile

Run as a desktop laboratory application from a known checkout and dedicated Python environment. Keep experiment files in a writable output directory and preserve source revision/package versions with results. No service installer, container, database migration, automatic updater, or production deployment procedure is supplied.

Starting Run attempts auxiliary services even for a small C37.118-only diagram. Use an isolated lab environment whose network interfaces and receiver endpoints you understand. Basic C37.118 experiments can run without raw-socket privileges.

Network footprint

Component	Default desktop behavior
PMU	UDP from an ephemeral source port to loopback/first PMU port, normally 4712
C37.118 proxy	Binds 127.0.0.1:4712; forwards to first Local PDC, normally 127.0.0.1:4713
Local PDC node	No listener created; represents an external destination
DNP3	UDP bind 0.0.0.0:20000, fallback ephemeral bind; target loopback port 20000
Modbus	TCP server loopback port 502, fallback 10502; local polling master
GOOSE	Attempts raw Ethernet publishing on autodetected Linux interface
REST	Not automatically started by Run; core default loopback:8080, manager configuration default all interfaces:8080
Syslog	Sends to configured collector only when configured/started; default loopback:514

Link labels and node ports for IEDs, switches, or State PDC do not imply additional listeners. Avoid using the same local endpoint for proxy input and forwarding output, which can cause a feedback loop.

Run procedure

1. Save current topology and record experiment settings.
2. Verify the selected input/output ports are free or belong to your intended receiver.
3. Start the receiver first if delivery validation matters.
4. Load/validate the topology while stopped.
5. Reset attack statistics and module state; review schedule and parameters.
6. Start a clean baseline and confirm proxy activity and receiver receipt.
7. Apply the intended change; collect application and receiver evidence.
8. Stop, export retained records, and save the final topology/settings.
9. Shut down any separately started API/capture/collector resources explicitly.

There is no autosave or automatic incident recovery. A topology is not a complete session backup. Keep JSON incident exports for detailed values; CSV is a summary and STIX is sampled.

Capacity and retention

Resource	Bound/default
Desktop C37.118 streams	1
PMU rate	30 data frames/s from GUI; constructor accepts 1-120
Active attack modules	1
Waveform buffer	120 samples per channel
Plot refresh	10 Hz for active plot
GUI log target	500 lines, with batched trimming
Incidents	Last 10,000 records
Recent packet summaries	Last 1,000 records
SSE subscriber queue	100 pending messages

At 30 records/s, the incident window is approximately 5.6 minutes and packet-summary window approximately 33 seconds. This is arithmetic from configured bounds, not a throughput benchmark. Extra records, delay, and loss change effective durations. There is no capture rotation or disk-quota manager.

Logs and diagnostics

GRIDSEC_LOG_LEVEL=DEBUG increases console logging. No rotating log file is configured. API startup logs the bearer token; redact it before sharing diagnostics.

PowerShell:

POWERSHELL

```
$env:GRIDSEC_LOG_LEVEL = 'DEBUG'
.\.venv\Scripts\python.exe main.py --demo *> gridsec-session.log
Get-NetUDPEndpoint | Where-Object LocalPort -in 4712,4713,20000
Get-NetTCPConnection -State Listen | Where-Object LocalPort -in 502,10502,8080,18080
```

Linux:

BASH

```
GRIDSEC_LOG_LEVEL=DEBUG .venv/bin/python main.py --demo > gridsec-session.log 2>&1
ss -lunp
ss -ltnp
```

Run socket inspection from another terminal while the application is active. These commands inspect listeners; they do not confirm application-level measurement acceptance.

Troubleshooting

Symptom	Likely explanation and action
No module named PyQt6	Wrong interpreter or incomplete install. Use the environment's Python and reinstall requirements there.
No visible Linux/WSL window	Missing display environment caused offscreen fallback. Confirm WSLg/display server and launch in a display-enabled session.
Qt XCB/plugin error	Missing/incompatible display libraries or environment. Verify distribution Qt dependencies and test offscreen separately.
Port already in use	Inspect the owner. Change the first PMU port or intended receiver port while stopped, then restart.
Packets plot but receiver sees nothing	Local PDC node is not a receiver. Check real listener, UDP/TCP match, destination, firewall/interface, and startup CFG-2 receipt.
Modified frames rejected downstream	Check codec channel counts and digital-word mismatch GS-08. Inspect outgoing bytes/configuration compatibility.
No attack effect	Confirm dropdown selection, enable toggle, schedule counter/window, and module-specific target. Reset state before a controlled repeat.
Replay initially looks clean	Buffer is still recording N frames; modifications begin afterward.
Drop does not make plot gaps	Plot consumes dropped records. Use dropped counter and receiver capture.
Delay looks clean	Expected classification; inspect elapsed delivery timing and throughput.
Capture/REST/Syslog button raises AttributeError	Missing convenience methods, GS-13. Use documented core interfaces; do not run patch helpers blindly.
PCAP empty or packet lengths zero	GUI lacks raw-byte delivery, GS-14; a writer alone is insufficient.
API protocol query empty	proto/protocol mismatch. Omit query filter and filter client-side.
API stops responding after SSE starts	Serial HTTP handler is occupied. Disconnect SSE and use polling.
API enable returns false	No attack hook installed. Desktop does not connect one.
API says simulation still running after Stop	Stale manager flag, GS-21. Check desktop/proxy state directly.
GOOSE fails on Windows	Linux socket family unavailable; use Linux lab for live Layer 2 publishing.
Modbus appears on 10502	Port 502 bind failed and server used its high-port fallback.
Long stop/restart or slow stream	Delay sleeps on proxy worker; workers stop asynchronously. Wait for termination before reusing ports.

Recovery and change management

Export records before closing whenever possible. After a crash, restart from a saved topology, reapply attack parameters/schedule, and begin a fresh baseline. In-memory incidents and module buffers are not recoverable.

For an update, preserve topology/export files and environment version records, stop the app, update the source in a controlled checkout, install requirements into a fresh environment, and run regression plus a relevant smoke check. To roll back, use the previous checkout/environment and revalidate the topology. Do not use workstation-specific deployment scripts as a release system.

Source: entry (main.py), runtime (gui/main_window.py), manager bounds (core/integration_manager.py), DNP3 (core/dnp3_simulator.py), Modbus (core/modbus_simulator.py).

CHAPTER 12

12 Security model and reporting

GridSec Sim is a local research and training application for authorized laboratory use. The current implementation has no production security certification or published supported-version/security-response policy.

Trust boundaries

- Topology input and attack parameters influence diagram state and, for selected fields, network destinations or transformations.
- Run attempts DNP3 and Modbus services plus a Linux raw Ethernet GOOSE publisher in addition to the C37.118 stream.
- The optional REST API can expose status, records, exports, and hook-based attack control. `/api/v1/status` is public even when bearer authentication is enabled.
- Exported measurements and recorded incidents are synthetic experimental data; desktop endpoint metadata is not a faithful packet capture.

Existing controls and gaps

The desktop binds its C37.118 proxy to loopback. The core API defaults to loopback and bearer auth, but the manager configuration default is `0.0.0.0`. DNP3 requests an all-interface UDP bind. Explicitly choose intended addresses and a controlled network environment.

Protocol streams and API HTTP are unencrypted. There is no user/role system, token expiry, request rate limit, durable audit trail, or hardened request validation. API tokens are logged at INFO. CORS is permissive. Syslog has no TLS or durable delivery queue. The cryptography dependency does not enable these controls automatically.

Keep local interfaces on loopback where practical. Protect logs containing tokens. Use a dedicated lab environment for Layer 2/raw-socket work instead of granting broad privileges to a general-purpose Python installation. Save work before opening untrusted/malformed topology files because the loader clears the canvas before rebuilding it.

Current source-confirmed gaps are tracked in [Known limitations](#); operational guidance is in [Operations](#).

Report a security issue

No dedicated security email or private reporting URL is configured in this checkout. Contact the project maintainer through an existing trusted private channel, or use private vulnerability reporting if it is enabled on the actual hosting repository. Do not invent or assume a contact address.

Provide the affected revision, minimal reproduction, impact, and redacted evidence. Exclude API tokens and real infrastructure identifiers. No response or remediation timeline is promised by this document.

Licensing status

Earlier project text says MIT, but an authoritative LICENSE file and copyright notice are absent. This document does not create a license or additional legal terms.

CHAPTER 13

13 Developer guide

Development environment

Create a dedicated environment using [Getting started](#), install `requirements.txt`, then run `test_regression.py`. The project is a source-tree application rather than an installed package: `main.py` inserts its directory into `sys.path`. There is no `pyproject.toml`, package build configuration, linter configuration, or dependency lockfile.

Run module self-checks with `python -m protocols.c37118` and similar module forms from the root to preserve import resolution. Existing shell deployment/test helpers contain fixed workstation paths; inspect them instead of assuming portability.

Source map

Files	Responsibility
<code>main.py</code>	Application bootstrap and process-level logging/error handling
<code>core/pmu_simulator.py</code> , <code>core/pdc_proxy.py</code>	Primary UDP producer and forwarding pipeline; optional core TCP proxy
<code>core/attack_engine.py</code> , <code>core/traffic_filter.py</code>	Transformation contract, scheduling, and packet eligibility
<code>core/packet_parser.py</code>	Protocol wrapper and global codec
<code>core/*_simulator.py</code>	Auxiliary GOOSE, DNP3, and Modbus traffic
<code>core/integration_manager.py</code>	Incident model, exports, Syslog/CEF, integration lifecycle
<code>core/rest_api.py</code> , <code>core/pcap_writer.py</code>	HTTP interface and synthesized capture writing
<code>protocols/</code>	Protocol-specific encoding/decoding helpers
<code>gui/main_window.py</code>	Component wiring, menus, scenarios, runtime lifecycle
<code>gui/canvas.py</code> , <code>gui/node_types.py</code>	Diagram interaction, persistence, node/link graphics
<code>gui/properties_panel.py</code> , <code>gui/attack_panel.py</code>	Property and attack forms
<code>gui/integration_panel.py</code>	Capture, syslog, REST, export and integration help tabs
<code>gui/waveform_viewer.py</code> , <code>gui/styles.qss</code>	Matplotlib plots and Qt theme

Core usage

Use `AttackEngine.set_attack()` with an `AttackType` enum and documented parameter dictionary. Use `set_schedule()` separately from attack parameters. Keep codec channel counts aligned across generation, parse, and rebuild; explicitly set the parser codec for the current single-process implementation.

The [offline example](#) demonstrates that contract with a known digital-word sentinel and validates the rebuilt result. The [API example](#) demonstrates manager lifecycle and a synthetic record without starting auxiliary network services.

PacketRecord does not hold outbound wire bytes. A capture integration must deliberately choose original or rebuilt bytes and pass them to `record_packet()`. Do not infer modified wire bytes from a modified dict's inherited `raw` value.

Add an attack module

1. Add an `AttackType` value and `BaseAttack` subclass.
2. Implement `get_params()`, `set_params()`, `reset()` when stateful, and `_apply_attack()` returning `AttackResult`.
3. Register the class in `ATTACK_CLASSES`.
4. Add an explicit form/order entry in `gui/attack_panel.py` and verify parameter synchronization.
5. Add meaningful checks for the transformation, no-op/drop behavior, reset, and any scheduling interaction.
6. Update [Attacks](#), relevant API/example metadata, and the changelog. Review export classification mappings without inventing a standards claim.

Prefer copying nested measurement arrays before modification. Validate numeric inputs and state transitions explicitly. Existing setters are not a uniform validation model to copy unquestioningly.

Add a node type

Register type/class, names, icons, colors, default ports, and level metadata in `node_types.py`. Add palette placement in `main_window.py`, configuration fields in `properties_panel.py`, and link defaults where relevant in `canvas.py`. Verify save/load with stable IDs and attachments.

A visual node does not become an executable device simply by registering it. Runtime construction, address mapping, callbacks, stop behavior, and protocol tests must be added separately if live behavior is intended.

Add or extend a protocol

Keep codecs independent of Qt. Define the supported wire subset, frame boundaries, units, error behavior, and representative valid/invalid vectors. Add a worker lifecycle appropriate to the transport and route callbacks through the relay for UI updates.

Decide whether the protocol observes original or modified measurements and whether it participates in attack execution. Connect nonempty raw bytes deliberately if capture is required. Verify with an independent decoder/receiver before claiming interoperability.

Extend integrations/API

Use manager `configure_*`(), `start()`, and `stop()` as the existing core lifecycle. Add missing GUI wrappers as ordinary source changes with targeted validation; `patch_mgr.py` rewrites source and is not a supported migration.

API callbacks execute outside the Qt thread. Marshal UI mutations through signals, validate request bodies, and return actual success. Before adding concurrent clients/streams, address process-global API and codec state, serial SSE handling, shared engine mutation, and shutdown ownership.

Engineering decisions for future scaling

The current diagram/runtime separation simplifies teaching but limits simulation fidelity. A multi-stream design should explicitly map node/link IDs to independent runtime contexts containing codec, filter, attack state, sockets, and capture policy. It should also define whether incident IDs, counters, and schedules are per stream or per session.

Keep those boundaries explicit before adding more UI switches. Confirm lifecycle ownership and failure reporting before claiming larger-scale support.

Documentation maintenance

When behavior changes, update the relevant guide/reference, machine-readable schemas/examples, limitations, and verification notes together. Re-run the relevant checks after code changes and record the resulting behavior. Do not claim test coverage from unchecked script banners.

Source: engine (core/attack_engine.py), nodes (gui/node_types.py), canvas (gui/canvas.py), manager (core/integration_manager.py), regression (test_regression.py).

CHAPTER 14

14 Contributing

Start with the [Developer guide](#) and [Known limitations](#). This repository does not yet define a formal maintainer roster, support SLA, contributor agreement, or release process.

Local workflow

1. Use a dedicated environment and run the current regression suite before changing behavior.
2. Keep the change focused and preserve unrelated work.
3. Test the actual behavior affected: codec bytes, attack outcome, UI synchronization, topology round-trip, or integration lifecycle.
4. Update user/reference documentation when a setting, schema, route, or supported workflow changes.
5. Provide a review description covering the problem, resulting behavior, validation, and remaining limitations.

See [Testing](#) for portable commands. Do not run `test_launch.sh` or deployment helpers without inspecting their source-copying behavior and fixed paths.

Bug reports

Include source revision, OS/Python/package versions, reproduction steps, expected/actual behavior, a minimal topology where relevant, and redacted logs. Distinguish GUI observations from independently verified network receipt. Identify whether a problem affects a documented limitation or a previously working path.

For sensitive security findings, follow [Security](#). Do not include API tokens, private receiver details, or unrelated production data in public reports.

Contribution boundaries

Use synthetic fixtures and loopback or explicitly authorized lab endpoints. Keep codecs usable without Qt. Preserve portable topology identifiers and compatibility deliberately. Avoid adding dependencies without documenting their purpose and installation impact.

Licensing remains unresolved until the maintainer supplies the authoritative LICENSE and copyright notice; the former README's MIT statement is not a substitute for that file.

CHAPTER 15

15 Testing and verification

Verification baseline

The following results were recorded during documentation verification using **Windows and Python 3.14.6**. They describe the checked application checkout, rather than an automatic guarantee for later code changes. Application source was not modified during that verification.

Installed GUI/numerical package versions in that environment were PyQt6 **6.11.0**, numpy **2.5.1**, and matplotlib **3.11.0**. These are observed test versions, not a new dependency lock or a recommendation to require those exact releases.

Check	Observed result	What it establishes
Existing <code>test_regression.py</code>	201/201 passed, 0 failed	Registered nodes/levels, canvas/property operations, ten attacks, schedule window, topology save/load and MitM attachment tests
C37.118 module self-check	Exit 0	Internal codec examples/assertions, including CRC and frame round-trips
DNP3 module self-check	Exit 0	Existing DNP3 framing/helper examples/assertions
Modbus module self-check	Exit 0	Existing Modbus frame/register examples/assertions
GOOSE module self-check	Exit 0	Offline GOOSE frame examples/assertions; no live Ethernet test
Attack engine self-check	Exit 0	Existing examples for all ten modules
Traffic filter self-check	Exit 0	Existing rule matching examples/assertions
Packet parser self-check	Exit 0	Existing parsing/rebuilding example
Offscreen MainWindow construction	Passed	Window can initialize; no live simulation was started by this check
Documentation topology in MainWindow	Passed	Three nodes load, validation has no errors/warnings, one intercepted link survives a save/load round-trip
Offline frame documentation example	Passed	56-byte original/rebuilt frame, Va 120 > 240 V, valid decode, unchanged frequency/timestamp/digital sentinel
Optional PCAP example	Passed	Two complete synthesized UDP records in a 252-byte PCAP
Local API example	Passed	Loopback startup, public status, auth, reads/exports, documented no-hook responses, bounded shutdown

The API check also reproduced the documented empty proto filter result, root 404, unavailable PCAP 404, and missing attack type 400. It did not open an SSE stream, invoke external collectors, or enable a remote attack.

These are observed checks, not a claim that every implementation path is correct. A number of gaps remain outside existing regression coverage, as listed in [Known limitations](#).

Run the existing regression suite

Windows:

POWERSHELL

```
.\.env\Scripts\python.exe test_regression.py
```

Linux/WSL:

BASH

```
.env/bin/python test_regression.py
```

The script creates an offscreen Qt application and returns nonzero if a `check()` fails. It is a standalone script, not a pytest suite. Some attack checks involve randomness and timing tolerances. Preserve failure output and investigate before repeating a failed run; repeated success alone does not explain a failure.

Run protocol/core self-checks

After activating your project environment, run from the repository root:

BASH

```
python -X utf8 -m protocols.c37118
python -X utf8 -m protocols.dnp3
python -X utf8 -m protocols.modbus
python -X utf8 -m protocols.goose
python -X utf8 -m core.attack_engine
python -X utf8 -m core.traffic_filter
python -X utf8 -m core.packet_parser
```

You can substitute the environment's full interpreter path for python. `-X utf8` avoids Windows console-encoding failures caused by Unicode success symbols. The first verification wrapper encountered such an output-encoding error; repeating the module checks with UTF-8 produced the results above. That was a reporting-environment issue, not a codec assertion failure.

These self-checks cover their embedded examples only. Printing "ALL TESTS PASSED" is not an external standards validation or a comprehensive test inventory.

Run documentation examples

BASH

```
python docs/examples/inspect_frame.py
python docs/examples/inspect_frame.py --pcap sample-frame.pcap
python docs/examples/serve_api.py --port 18080 --duration 10
```

The PCAP example requires a new filename and opens no network sockets. The API example opens one loopback HTTP listener, prints a local token, and serves one synthetic incident. Use another terminal to query it while running. Omit `--duration` for an interactive session terminated with `Ctrl+C`.

Manual smoke checks for functional changes

Change area	Useful validation
Topology/editor	Add/edit/remove nodes, validate, save/reload, inspect stable IDs and attachment restoration
Attack behavior	Verify exact transformed values, no-op/drop flags, boundaries, schedule indexing and reset behavior
Codec/parser	Check encoded length, CRC, channel counts, timestamp preservation, malformed/short input, and independent decoder acceptance
GUI runtime	Start/stop on isolated endpoints; inspect actual worker/socket state and receiver receipt
Capture	Confirm nonempty selected wire bytes, packet count, headers and independent PCAP parsing
API	Check auth, bad payloads, hook outcomes, actual state transitions, client concurrency and shutdown
Integration lifecycle	Check bind/open failure, stop/restart, resource cleanup and missing GUI wiring

A manual GUI run should start with Clean Baseline, then a controlled magnitude/frequency change, then a reset and finite schedule. For packet loss/delay, use receiver-side timing/counts rather than expecting the waveform alone to show delivery behavior.

Historical shell scripts

File	Current caveat
test_all.sh	Assumes ~/GridSecSim and venv/; limited module checks
test_launch.sh	Copies files from a fixed workstation path before launching a check
deploy_and_test.sh, deploy_integration.sh	Copy/overwrite files between fixed Windows/WSL locations
patch_mgr.py	Rewrites manager source; not a test or supported migration

Use the direct Python commands above for portable verification. Do not run source-copying scripts just to obtain a test result.

Documentation artifact checks

The documentation review checks local Markdown targets, JSON syntax/internal references, Python example syntax, API route coverage against the handler, and example topology references. The topology example is also loaded through the actual canvas.

The JSON Schema and OpenAPI files are reference artifacts, not newly installed validators or API behavior changes. Full third-party schema/OpenAPI conformance validation is not claimed. Likewise, STIX export has not been checked with a strict external validator.

Not verified in this review

- Live openPDC or another external PDC receiving modified frames.
- Live Linux/WSL raw Ethernet GOOSE publishing.

- External SIEM/syslog collector receipt or named vendor interoperability.
- Full standards conformance, long-duration stability, measured capacity, or concurrent API/SSE load.
- Linux/WSL installation from a clean machine, GUI visual behavior across display stacks, or a packaged release.

Source: regression suite (`test_regression.py`), codec self-checks (`protocols/c37118.py`), engine self-check (`core/attack_engine.py`), [documentation examples](#).

CHAPTER 16

16 Known limitations

These findings describe the documented application implementation. They are implementation constraints, not speculative vulnerabilities or a list of completed fixes.

Runtime and topology

ID	Finding	Practical consequence
GS-01	Desktop creates one PMU/proxy from the first PMU and Local PDC.	Multiple nodes/links do not create simultaneous independent flows.
GS-02	Runtime fixes loopback PMU input, UDP, 30 frames/s, 50 Hz, 120 V, and PMU ID 1.	PMU IP, reporting rate, ID and node transport edits do not control those settings.
GS-03	Threat Agent attachments/scope are not consulted by the runtime engine.	Attacks can run without an attached agent; selected-link scope is not enforcement.
GS-04	Saved agent attack metadata is not loaded into the engine.	Loading a topology does not arm its saved attack policy.
GS-05	Engine counters/state/schedule survive Stop/Run.	Reset explicitly for comparable runs; a finite window may already be over.
GS-06	Ordinary Open can replace a running diagram; scenario selection can still alter attacks when demo loading is blocked.	Stop before loading diagrams or scenarios.
GS-07	Top-level Attacks menu ignores its selected module argument.	Use the Attacks-tab dropdown and enable action.

Sources: MainWindow (gui/main_window.py), AttackPanel (gui/attack_panel.py), Canvas (gui/canvas.py).

Packet integrity and observation

ID	Finding	Practical consequence
GS-08	Parser global codec has zero digital words; PMU generates one; proxy does not call <code>set_codec()</code> .	Modified frames can omit the digital word and no longer match initial CFG-2 layout.
GS-09	Rebuild exceptions fall back to original raw bytes; modified classification remains.	An attacked record alone does not prove changed outgoing bytes.
GS-10	Records precede forwarding and plots accept dropped/non-data records.	Plots and dashboard do not verify receiver delivery; startup artifacts and loss without gaps are possible.
GS-11	Delay returns no content-modified/drop flag.	Delayed packets appear clean and do not create attacked SSE/syslog events.
GS-12	Auxiliary protocols use original measurements and independent singleton simulators.	C37.118 attack effects do not propagate to GOOSE/DNP3/Modbus.

Sources: Parser (core/packet_parser.py), Proxy (core/pdc_proxy.py), Engine (core/attack_engine.py), Viewer (gui/waveform_viewer.py).

Integration and API

ID	Finding	Practical consequence
GS-13	Manager lacks panel-used convenience methods: <code>start_pcap_only</code> , <code>stop_pcap_only</code> , <code>start_syslog_only</code> , <code>stop_syslog_only</code> , <code>test_syslog</code> , <code>start_rest_only</code> , <code>stop_rest_only</code> , <code>disable_rest_auth</code> .	Affected integration buttons can raise <code>AttributeError</code> . Core configure/start/stop interfaces remain available.
GS-14	GUI reads missing <code>PacketRecord.raw_bytes</code> .	PCAP receives empty payloads; recent packet lengths are zero. Fixing only Start Capture does not fix capture data.
GS-15	Desktop records synthetic addresses/ports and only C37.118 callbacks.	Exports do not faithfully represent live endpoint addresses or all auxiliary traffic.
GS-16	Desktop does not set remote attack hooks.	Enable can return false; disable can report success without changing the engine.
GS-17	API proto filtering reads proto while incidents contain <code>protocol</code> .	Standard-manager filtered incident queries return empty results.
GS-18	REST uses serial <code>HTTPServer</code> with long-lived SSE.	SSE can block other requests and delay server shutdown.
GS-19	Root URL is stripped to an empty path.	Use <code>/api/v1/docs</code> ; <code>/</code> is not a working help route.
GS-20	API state/auth are process-global; limits and POST payloads lack consistent validation.	Independent server instances are not isolated; clients can receive 500s or broken connections for bad input.
GS-21	Manager <code>sim_running</code> is not reset by desktop Stop; attack snapshot is counters only.	API status can be stale and <code>/attacks/current</code> is not a complete active-policy description.
GS-22	No explicit integration-manager stop in <code>MainWindow</code> close; PCAP open failures may be stored without raising.	Explicit lifecycle and readiness checks are needed in embedded integrations.
GS-23	STIX uses custom fields, static mappings, and limited samples.	Validate/adapt before importing; do not treat it as a conformance-certified or complete evidence package.

Sources: `IntegrationPanel` (`gui/integration_panel.py`), `IntegrationManager` (`core/integration_manager.py`), REST server (`core/rest_api.py`), PCAP writer (`core/pcap_writer.py`).

Platform and project maturity

- GOOSE uses `LINUX AF_PACKET`; native Windows live publishing is unavailable and its worker may raise an `unsupported-family` exception.
- GUI startup's "running" messages do not replace socket readiness checks; startup uses fixed sleeps and several workers initialize asynchronously.
- No multi-platform live-network interoperability suite, protocol certification, production benchmark, load test, or packaged installer is established here.
- Existing `test_launch.sh` and deployment scripts contain workstation-specific paths and copy/overwrite source. They are not portable test/release commands.
- No dependency lockfile, CI workflow, durable incident store, TLS, account/role system, or authoritative `LICENSE` file is included.

See [Testing](#) for what has actually been verified. A future fix should update this page, affected workflow documentation, and the corresponding validation evidence together.

CHAPTER 17

17 Glossary

Term	Meaning in this project
Attack engine	Applies one selected transformation to eligible decoded data frames
BCU	Bay Control Unit; a diagram node representing bay switching/control
C37.118	Synchrophasor communication format targeted by the primary codec/runtime
CEF	Common Event Format used by the Syslog exporter
CFG-2	Configuration frame describing measurement channels and reporting setup
CRC	Checksum used to detect frame corruption; recalculation does not authenticate a sender
CT/VT	Current/voltage transformer, represented as a process-level sensor node
DNP3	Protocol family represented by framing helpers and a limited UDP simulator
Frame	Encoded protocol message, or its decoded dictionary when stated explicitly
GOOSE	IEC 61850-associated event-message format carried directly over Ethernet
HMI	Human-machine interface; station diagram asset in GridSec Sim
IDCODE	Identifier carried in C37.118 headers
IED	Intelligent Electronic Device; protection relay diagram type
Incident	Manager record for any processed callback, including clean traffic
Integration manager	Owns retained records, exporters, and optional external interfaces
Local PDC	Diagram destination used by the desktop proxy; does not start a receiver
MitM	Man in the middle; here, an explicitly configured local forwarding proxy
Modbus TCP	Register-oriented protocol served/pollled by the auxiliary simulator
PacketRecord	Internal proxy record containing original/modified dicts and classification
PCAP	Packet capture file format; core writer creates synthesized network frames
PDC	Phasor Data Concentrator, an intended downstream measurement receiver
Phasor	Magnitude and phase angle representation of a periodic quantity
PMU	Phasor Measurement Unit; here, a synthetic measurement generator
ROCOF / DFREQ	Rate of change of frequency, represented in Hz/s
RTU	Remote Terminal Unit; gateway diagram type
SCADA	Supervisory Control and Data Acquisition
SIEM / SOAR	Security event analysis / response automation systems that may consume exports

Term	Meaning in this project
SSE	Server-Sent Events, the API's optional live HTTP event stream
State PDC	Higher-level diagram node with a regional uplink indicator
STIX	Structured threat-information representation targeted by the experimental exporter
Threat Agent	Canvas node recording interception metadata; not an independent runtime engine
Topology	Saved diagram of nodes, links, and metadata
Traffic filter	Rules selecting frames for attack processing, not a network firewall
WSL	Windows Subsystem for Linux; a possible Linux runtime/display environment

For support boundaries, consult [Protocols](#) and [Known limitations](#).

CHAPTER 18

18 Changelog

This file records verified project history and documentation changes. It does not imply that a packaged release or tag exists.

Unreleased - documentation

- Replaced the introductory README with a current project entry point.
- Added user, installation, architecture, configuration, attack, protocol, API, data-format, integration, operations, development, and testing documentation.
- Added OpenAPI and topology-schema reference artifacts and runnable documentation examples.
- Added a detailed PDF manual and an editable Overleaf LaTeX project, with the project GitHub link.
- Recorded source-confirmed implementation limits, security model, contribution workflow, and unresolved licensing status.

Application-reported version remains 1.0.0; this documentation work does not change runtime behavior.

Guided simulation and desktop integrations

Git subject: feat: add guided simulation workflow and desktop integrations.

The baseline includes the bundled MitM demo, guided scenarios, validation, packet scheduling, portable topology saves with attachment restoration, and desktop integration-related work. These features are described according to current wiring and limitations in the documentation, not solely by the commit title.

The earlier WHAT_WAS_ADDED.md (WHAT_WAS_ADDED.md) is retained as historical context.

APPENDIX

19 Reference files

The five companion files below are embedded in this PDF. Extract them using a reader that supports PDF attachments. Python examples still require the project source and its documented environment; they are not standalone binaries.

Attached file	Purpose
openapi.json	Machine-readable HTTP interface
topology-v2.schema.json	Portable topology v2 schema
topology-v2.json	Three-node MitM example
inspect_frame.py	Offline frame and sample-PCAP example
serve_api.py	Loopback API example

Using the reference files

These attachments reproduce the documentation artifacts present when this PDF was compiled. The OpenAPI and JSON Schema files describe interfaces; they do not change or validate runtime behavior automatically. They are subject to the same documented limitations as the human-readable references.

Project GitHub: [neerajjayesh/GridSec-project](https://github.com/neerajjayesh/GridSec-project).

Document navigation

Internal guide links resolve to chapters in this manual. Links to schema/example artifacts lead to this appendix. Source code references retain repository paths for readers working with the checkout. No external vendor interoperability or certification is implied by the export formats.

APPENDIX

20 Practical examples

The complete examples below accompany the installation, data-format, and API guides. Use the copies under docs/examples in the repository, or extract the identical PDF attachments into that directory. The Python scripts locate project imports relative to their own file paths and require the installed project dependencies.

20.1 Save a three-node topology

This version 2 topology contains one PMU, one PDC, and one Threat Agent assigned to the PMU-to-PDC link. Open docs/examples/topology-v2.json through the topology workflow in the User guide, then inspect and validate the diagram. Its interception association describes the diagram; it does not establish per-link network routing. Runtime behavior and persistence limits remain those described in Chapters 3 and 10.

TOPOLOGY-V2.JSON / JSON

```
{
  "format": "GridSecSim",
  "version": 2,
  "nodes": [
    {
      "id": "10000000-0000-4000-8000-000000000001",
      "node_type": "PMU",
      "label": "PMU-Example",
      "ip": "127.0.0.1",
      "port": 4712,
      "proto": "UDP",
      "level": 1,
      "reporting_rate": 30,
      "idcode": 1,
      "running": false,
      "active": false,
      "x": -180,
      "y": 300
    },
    {
      "id": "10000000-0000-4000-8000-000000000002",
      "node_type": "PDC",
      "label": "PDC-Example",
      "ip": "127.0.0.1",
      "port": 4713,
      "proto": "UDP",
      "level": 2,
      "buffer_timeout": 100,
      "pmu_input_count": 1,
      "x": 280,
      "y": 100
    },
    {
      "id": "10000000-0000-4000-8000-000000000003",
      "node_type": "THREAT_AGENT",
      "label": "MitM-Example",
      "ip": "127.0.0.1",
      "port": 0,
      "proto": "UDP",
      "level": -1,
      "active": true,
      "x": 50,
      "y": 200
    }
  ],
  "links": [
```

TOPOLOGY-V2.JSON / JSON / CONTINUED

```
{
  "src_id": "10000000-0000-4000-8000-000000000001",
  "dst_id": "10000000-0000-4000-8000-000000000002",
  "protocol": "C37.118",
  "threat_agent_ids": [
    "10000000-0000-4000-8000-000000000003"
  ]
}
```

20.2 Transform a frame offline

Run the script below from the repository root with your configured Python environment. It constructs one 56-byte C37.118 data frame, applies SCALE with a factor of 2.0, rebuilds it, and checks the decoded result. The first phasor magnitude changes from 120.0 to 240.0 V; frequency, timestamp, and the digital sentinel remain unchanged. No network sockets are opened.

COMMANDS

```
python docs/examples/inspect_frame.py
python docs/examples/inspect_frame.py --pcap sample-frame.pcap
```

The first command reports PASS if its assertions succeed. The optional second command creates a two-record synthetic PCAP and refuses to overwrite an existing file. These records contain generated Ethernet/IP/UDP headers; they are not interface captures.

INSPECT_FRAME.PY / PYTHON

```
"""Offline C37.118 transformation example. Run from any directory.
```

```
python docs/examples/inspect_frame.py
python docs/examples/inspect_frame.py --pcap sample-frame.pcap
```

```
No network sockets are opened. Optional capture output contains synthesized
Ethernet/IP/UDP headers, not packets observed on a network interface.
"""
```

```
import argparse
import math
from pathlib import Path
import sys
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(PROJECT_ROOT))
```

```
from core.attack_engine import AttackEngine, AttackType
from core.packet_parser import parse_frame, rebuild_frame, set_codec
from core.pcap_writer import PcapWriter
from protocols.c37118 import C37118Codec
```

```
def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--pcap', type=Path, help='Create a new sample PCAP file')
    args = parser.parse_args()
    if args.pcap and args.pcap.exists():
        parser.error('Capture path already exists; choose a new output filename.')
```

```
# Explicitly align parser/generator counts, including the digital word.
codec = C37118Codec(idcode=1, num_phasors=3, num_analog=1, num_digital=1)
set_codec(codec)
```

INSPECT_FRAME.PY / PYTHON / CONTINUED

```

original_raw = codec.encode_data_frame(
    phasors=[(120.0, 0.0), (119.5, -2 * math.pi / 3),
              (120.5, 2 * math.pi / 3)],
    freq=50.0, dfreq=0.0, analog=[500.0], digital=[0xA5A5],
    soc=1700000000, fracsec=0,
)
original = parse_frame(original_raw)
assert original is not None and original['frame_type'] == 'data'

engine = AttackEngine()
engine.set_attack(AttackType.SCALE, {'scale_factor': 2.0})
modified, changed, dropped = engine.apply(original)
modified_raw = rebuild_frame(modified)
decoded = codec.decode_data_frame(modified_raw)

assert changed and not dropped
assert decoded is not None, 'Rebuilt frame must decode with a valid CRC'
assert len(original_raw) == len(modified_raw) == 56
assert math.isclose(decoded.phasors[0][0], 240.0)
assert math.isclose(decoded.freq, 50.0)
assert decoded.digital == [0xA5A5], 'Digital word must survive rebuilding'
assert decoded.soc == 1700000000
print('PASS: 56-byte frame; Va 120.0 -> 240.0 V; frequency, time, digital word preserved.')

if args.pcap:
    writer = PcapWriter(str(args.pcap.resolve()))
    try:
        if not writer.is_active:
            raise RuntimeError(writer.error or 'PCAP writer did not become active')
        for offset, payload in enumerate((original_raw, modified_raw)):
            writer.write_udp('192.0.2.1', '192.0.2.2', 49000, 4712,
                             payload, ts=1700000000.0 + offset)
        assert writer.packet_count == 2
    finally:
        writer.close()
    print(f'Created two-packet sample: {args.pcap.resolve()}')
return 0

if __name__ == '__main__':
    raise SystemExit(main())

```

20.3 Inspect the local REST API

This bounded example records one synthetic attacked packet and starts the API on loopback. It does not start the protocol simulators. Run the first command in one terminal, copy the displayed bearer token, and use it in a second terminal while the example is running. The token is specific to the example server.

COMMANDS / WINDOWS POWERSHELL

```

python docs/examples/serve_api.py --port 18080 --duration 120

# Second terminal: replace PRINTED_TOKEN with the displayed token.
curl.exe -H "Authorization: Bearer PRINTED_TOKEN" http://127.0.0.1:18080/api/v1/status

```

On Linux, use curl in place of curl.exe. The status endpoint is available while the simulation remains stopped; remote attack hooks are unset. See Chapter 8 for response fields and other routes. The server exits after 120 seconds, or on Ctrl+C. Disconnect any event-stream clients before shutdown.

SERVE_API.PY / PYTHON

```
"""Serve one synthetic GridSec incident on loopback; no protocol simulators.
```

```
python docs/examples/serve_api.py
python docs/examples/serve_api.py --port 18081 --duration 10
```

```
The printed token grants access to this example server. Remote attack hooks
are deliberately unset. Disconnect SSE clients before shutting the server down.
"""
```

```
import argparse
from pathlib import Path
import sys
import time
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(PROJECT_ROOT))
```

```
from core.integration_manager import IntegrationManager
from core.pdc_proxy import PacketRecord
```

```
def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--port', type=int, default=18080)
    parser.add_argument('--duration', type=float, default=0,
                        help='Exit after N seconds; zero waits for Ctrl+C')
    args = parser.parse_args()
    if not 1024 <= args.port <= 65535:
        parser.error('Choose an unprivileged port from 1024 through 65535.')
    if args.duration < 0:
        parser.error('Duration must be nonnegative.')

    manager = IntegrationManager()
    manager.record_packet(
        PacketRecord(
            timestamp=time.time(),
            original={'frame_type': 'data', 'freq': 50.0, 'phasors': [[120.0, 0.0]]},
            modified={'frame_type': 'data', 'freq': 50.0, 'phasors': [[240.0, 0.0]]},
            status='attacked', attack_type='SCALE',
            description='Synthetic documentation example',
        ),
        protocol='C37.118', src_ip='192.0.2.1', dst_ip='192.0.2.2',
    )
    manager.configure_rest_api(host='127.0.0.1', port=args.port)
    result = manager.start()
    if not result.get('rest_api', '').startswith('listening on port '):
        manager.stop()
        print(f'Server failed to start: {result}', file=sys.stderr)
        return 1

    try:
        print(f'URL: http://127.0.0.1:{args.port}/api/v1/status', flush=True)
        print(f'Bearer token: {manager.api_key}', flush=True)
        print('One synthetic incident; simulation stopped; attack hooks unset. Ctrl+C to exit.', flush=True)
        deadline = time.monotonic() + args.duration if args.duration else None
        while deadline is None or time.monotonic() < deadline:
            time.sleep(0.1)
    except KeyboardInterrupt:
        pass
    finally:
        manager.stop()
    return 0

if __name__ == '__main__':
    raise SystemExit(main())
```